

Final Project: The Disney Corporation Strategy Decision

Team A: Anggo Irfan, Balqis Kanaga, Georgia Kingman, Sabrina Vasquez

Patrick Tyson, Vivek Dhotrekar

AD654 O2 Marketing Analytics

Boston University Metropolitan College

August 11, 2025

Table of Contents

- **Executive Summary**
- **I. Tableau** – Balqis
- **II. Summary Stats** – Irfan & Georgia
- **III. Segmentation & Targeting** – Sabrina
- **IV. Conjoint** – Balqis
- **V. Strategic Memo** – Georgia
- **VI. Forecast** – Irfan
- **VII. Classification** – Sabrina
- **VIII. A/B** – Irfan
- **IX. Final Conclusion** – Balqis & Georgia

Executive Summary

In this project, we applied a range of marketing analytics methods such as Tableau visualizations, summary statistics, segmentation and targeting, conjoint analysis, forecasting, classification modeling, and A/B testing to evaluate Disney's current performance and identify actionable growth opportunities. Our analysis aimed to highlight Disney's strengths, pinpoint areas of improvement, and guide future investments with data-backed recommendations.

From the segmentation and targeting analysis, we identified four key audience clusters: High Spenders, Budget Families, Park Lovers, and Wealthy Shoppers. Each group has unique behaviors and spending patterns. High Spenders and Wealthy Shoppers respond well to luxury offerings, VIP experiences, and exclusive merchandise, while Park Lovers value multivisit packages and event access. Budget Families are more price sensitive and respond best to discounts, seasonal promotions, and family bundles. These insights

allow us to tailor marketing campaigns, pricing, and product offerings to maximize appeal and retention for each segment.

The classification model, designed to predict repeat cruise bookings, revealed that households who booked a park package with their cruise were less likely to return. At the same time, families with more children and those closer to embarkation ports were more likely to book again. Ports like San Diego and Vancouver showed higher return rates, suggesting opportunities for localized marketing. With 61% accuracy, the model offers a strong foundation for targeted retention strategies, particularly by focusing on high likelihood repeat customers and avoiding offerings that discourage future bookings.

Additional analyses reinforced these findings. The conjoint analysis showed that a specific set of hotel amenities, such as Best in Class Wi-Fi, flexible check-in, full buffet breakfast, and VIP shopping access, maximizes guest satisfaction while staying within budget. Forecasting projected moderate post-pandemic recovery, but still below pre-2020 profit peaks, underscoring the need for strategic investment. A/B testing identified high-performing merchandise styles, like MickeySuit, Cinderella, and Mermaid, that should be prioritized in production and marketing.

Overall, we recommend prioritizing the MagicBand rollout to enhance the guest experience and data collection, expanding Aulani with a phased approach to diversify revenue, and continuing to invest in IP-based park expansions to maintain a competitive advantage. By combining imaginative experiences with precise, segment-specific targeting and retention strategies, Disney can secure long-term growth and sustain its market leadership.

I. Tableau

Tableau Dashboard Link:

<https://public.tableau.com/app/profile/b.hannan/viz/FinalAssignment-Graphs/FinalProject?publish=yes>

The **Total Gross Revenue by Genre** chart shows that Adventure, Action, and Comedy have earned the most overall, with Adventure bringing in a little over **\$16 billion**, Action close to **\$14 billion**, and Comedy just over **\$9 billion**. Genres such as Family, Black Comedy, and Concert/Performance are at the bottom, and several others, including Horror, Crime, Western, Biography, and Documentary, have earned less than **\$1 billion** each. These results highlight which genres have historically brought in the most revenue and which tend to underperform. This suggests that continuing to produce movies in the most successful categories could lead to stronger financial results, while investments in lower-performing genres may need to be reassessed to ensure resources are used effectively.

The **Inflation-Adjusted Gross Revenue by Release Year** chart shows two big peaks in Disney's earnings. The first was in the early decades, when adjusted revenue was close to **\$4 billion**, driven by some of Disney's most iconic releases. Following that, the numbers fluctuated a lot. Between **1995 and 2005**, the number of movies released was high compared to the rest of the years, but total revenue didn't increase much, meaning many of those films didn't perform as strongly as expected. The second peak was between **2015 and 2020**, when revenue was again close to **\$4 billion**, but with far fewer movies. This shows that films from this period were especially successful.

The **top-grossing movies** show that the biggest hits tend to come from major franchises like Star Wars, Avengers, Black Panther, and Avatar. These are mostly superhero, fantasy, or action films with huge fan bases and familiar worlds that people love coming back to. They attract all kinds of audiences, perform well internationally, drive merchandise sales, and often get people to see them more than once in theaters. Of course, it's still important to make a variety of films, but the consistent success of these blockbusters shows that keeping them in the mix, and potentially developing new franchises with that same wide appeal, could be a smart way to keep box office numbers strong.

The **Distribution of Movies by MPAA Rating** chart shows that most Disney films are rated PG-13, PG, or R. This pattern suggests a deliberate effort to appeal to broad audiences, with the intention of balancing family-friendly releases with titles that attract older viewers. Maintaining this mix allows Disney to keep a strong presence in both family entertainment and blockbuster markets. Continuing to focus on these ratings could help sustain audience reach across different age groups while still allowing room for variety in the lineup.

II. Summary Stats

```
In [2]: # SUMMARY STATS

import pandas as pd
import numpy as np

df = pd.read_csv("../AD 652 Final Project/disney_movies.csv")
df.head()
```

Out [2]:

	movie_title	release_date	genre	mpaa_rating	total_gross	inflation_adjusted_g
0	A Complete Unknown	12/25/2024	Biography	R	74998243	74998243
1	Mufasa: The Lion King	12/20/2024	Animation	PG	254567693	254567693
2	Moana 2	11/27/2024	Animation	PG	460405297	460405297
3	A Real Pain	11/1/2024	Comedy	R	8344978	8344978
4	Alien: Romulus	8/16/2024	Horror	R	105313091	105313091

In [3]:

Overall summary (numeric only)
print("=== Numeric Summary ===")
display(df.describe())

=== Numeric Summary ===

	total_gross	inflation_adjusted_gross
count	8.310000e+02	8.310000e+02
mean	6.550697e+07	1.277468e+08
std	1.084570e+08	2.477998e+08
min	2.735000e+03	5.263617e+03
25%	9.036157e+06	1.765426e+07
50%	2.669285e+07	5.454011e+07
75%	7.119446e+07	1.348556e+08
max	9.366622e+08	4.028550e+09

In [4]:

Overall summary (categoricals only)
print("=== Categorical Summary ===")
for col in df.select_dtypes(include=["object"]).columns:
 print(f"\nColumn: {col}")
 print(df[col].value_counts())

=== Categorical Summary ===

Column: movie_title

movie_title

Snow White and the Seven Dwarfs 1

A Complete Unknown 1

Mufasa: The Lion King 1

Moana 2 1

A Real Pain 1

..

The Boogeyman 1

Elemental 1

Indiana Jones and the Dial of Destiny 1

Theater Camp 1

Haunted Mansion 1

Name: count, Length: 831, dtype: int64

Column: release_date

release_date

12/25/1997 3

7/17/1992 2

9/18/1992 2

2/3/1993 2

4/16/1993 2

..

8/13/1942 1

6/20/1941 1

11/13/1940 1

2/9/1940 1

11/27/2024 1

Name: count, Length: 799, dtype: int64

Column: genre

genre

Comedy 262

Drama 146

Adventure 144

Action 90

Animation 28

Documentary 25

Thriller/Suspense 24

Romantic Comedy 23

Horror 17

Musical 16

Crime 14

Biography 14

Western 7

Black Comedy 3

Concert/Performance 2

Family 2

Name: count, dtype: int64

Column: mpaa_rating

mpaa_rating

PG-13 222

PG 221

```

R          215
G          109
Approved    8
Not Rated   4
Name: count, dtype: int64

```

```

In [5]: # Missing values
print("\n=== Missing Values ===")
print(df.isna().sum())

```

```

=== Missing Values ===
movie_title          0
release_date         0
genre               14
mpaa_rating          52
total_gross          0
inflation_adjusted_gross  0
dtype: int64

```

```

In [6]: # Correlation matrix for numeric columns
print("\n=== Correlation Matrix ===")
display(df.select_dtypes(include=[np.number]).corr())

```

```

=== Correlation Matrix ===

```

	total_gross	inflation_adjusted_gross
total_gross	1.000000	0.602914
inflation_adjusted_gross	0.602914	1.000000

```

In [7]: # Group by Genre: mean & total gross
if "genre" in df.columns:
    print("\n=== Gross by Genre ===")
    display(
        df.groupby("genre")[["total_gross", "inflation_adjusted_gross"]]
        .agg(["count", "mean", "sum"])
        .sort_values(("total_gross", "sum"), ascending=False)
    )

```

```

=== Gross by Genre ===

```

genre	total_gross			inflation_adjusted		
	count	mean	sum	count	mean	
Adventure	144	1.126642e+08	16223649336	144	2.074743e+08	2.9876
Action	90	1.522379e+08	13701409406	90	2.023909e+08	1.8215
Comedy	262	3.547757e+07	9295123575	262	7.803175e+07	2.0444
Animation	28	1.736410e+08	4861947157	28	3.530482e+08	9.8853
Drama	146	3.000113e+07	4380164361	146	6.876664e+07	1.0039
Thriller/Suspense	24	5.861694e+07	1406806519	24	1.055570e+08	2.5333
Musical	16	7.233026e+07	1157284155	16	5.725531e+08	9.1608
Romantic Comedy	23	5.009595e+07	1152206855	23	9.235459e+07	2.1241
Horror	17	3.893853e+07	661954968	17	6.310154e+07	1.0727
Crime	14	3.075440e+07	430561580	14	4.722365e+07	6.6113
Western	7	5.128735e+07	359011459	7	8.742732e+07	6.1199
Biography	14	2.066201e+07	289268105	14	2.956570e+07	4.1391
Documentary	25	7.747982e+06	193699545	25	1.132149e+07	2.8303
Concert/Performance	2	5.172823e+07	103456466	2	7.547366e+07	1.5094
Black Comedy	3	3.251440e+07	97543212	3	6.018195e+07	1.8054
Family	2	3.875000e+06	7750000	2	2.537559e+07	5.0757

```
In [8]: # Group by MPAA Rating: mean & total gross
if "mpaa_rating" in df.columns:
    print("\n=== Gross by MPAA Rating ===")
    display(
        df.groupby("mpaa_rating")[["total_gross", "inflation_adjusted_gross"]
          .agg(["count", "mean", "sum"])
          .sort_values(("total_gross", "sum"), ascending=False)
        )
```

=== Gross by MPAA Rating ===

mpaa_rating	total_gross			inflation_adjusted_gross		
	count	mean	sum	count	mean	sum
PG-13	222	9.447270e+07	20972938663	222	1.332607e+08	2.958387e+10
PG	221	7.987051e+07	17651383456	221	1.240144e+08	2.740719e+10
G	109	8.092969e+07	8821336363	109	2.762317e+08	3.010925e+10
R	215	2.691639e+07	5787024605	215	5.022291e+07	1.079792e+10
Approved	8	2.425271e+07	194021650	8	3.696216e+08	2.956973e+09
Not Rated	4	3.820524e+07	152820979	4	3.577936e+08	1.431174e+09

Dataset Description

- The most frequent genres are Comedy (262 titles), Drama (146), and Adventure (144), while PG-13 and PG films dominate in ratings.
- Adventure films generate the highest total gross (about **\$16.2 billion**), followed by Action (about **\$13.7 billion**) and Comedy (about **\$9.3 billion**), whereas Animation has the highest average gross per title (about **\$173 million**), showing strong per-film profitability.
- PG-13 movies lead total gross (about **\$20.97 billion**), indicating broad audience appeal, with PG and G films also performing strongly.
- The correlation between total gross and inflation-adjusted gross is moderate ($r \approx 0.60$), suggesting that time of release impacts revenue.
- Missing data in genre (14 films) and MPAA rating (52 films) should be addressed for better analysis.

These insights highlight that future production and marketing could focus on Adventure, Action, and high-quality Animation films with PG or PG-13 ratings to maximize audience reach and revenue potential.

III.Segmentation and Targeting

```
In [1]: # Libraries
import pandas as pd, numpy as np
import seaborn as sns, matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

```
sns.set_theme(style="whitegrid")
fmt_dollar = FuncFormatter(lambda x, pos: f'${x:,.0f}')
```

```
In [13]: # Loading the file
df_family_segments = pd.read_csv("family_segments.csv")
df_family_segments.columns = [c.strip() for c in df_family_segments.columns]
```

```
In [3]: # Show file
print(df_family_segments.shape)
df_family_segments.head()
```

(9250, 12)

```
Out[3]:
```

	hhold_ID	est_net_worth	est_inc	domestic	number_visits	number_children	oldest_child_age
0	1	7525	7825	1	0	2	
1	2	639815	449775	1	1	2	
2	3	288847	204750	1	6	3	
3	4	191791	136730	1	1	1	
4	5	96527	72165	0	3	0	

```
In [14]: # Data Preparation for the k-means clustering

# Numeric variables for clustering
numeric_cols = [
    'est_net_worth', 'est_inc', 'number_visits', 'number_children',
    'oldest_child_age', 'online_merch_avg', 'est_annual_travel',
    'est_annual_leisure', 'est_hhold_FICO'
]

# Categoricals for later profiling
cat_cols = ['domestic', 'dis_plus_sub']

# Fill missing values with the median
X = df_family_segments[numeric_cols].apply(pd.to_numeric, errors='coerce')
X = X.fillna(X.median())

# Standardize (z-scores) for k-means
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Prepared array shape:", X_scaled.shape)
```

Prepared array shape: (9250, 9)

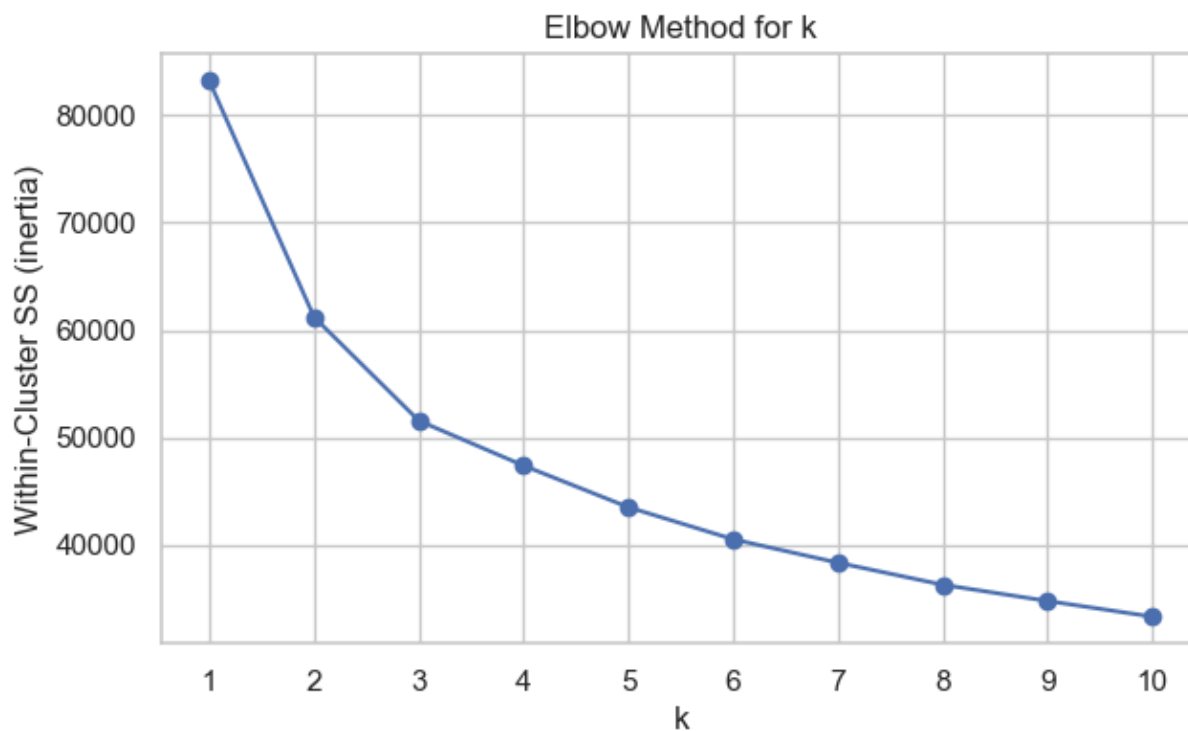
```
In [5]: # Elbow to find the number of clusters
Ks = range(1, 11)
inertias = []
for k in Ks:
    km = KMeans(n_clusters=k, n_init=25, random_state=42)
    km.fit(X_scaled)
```

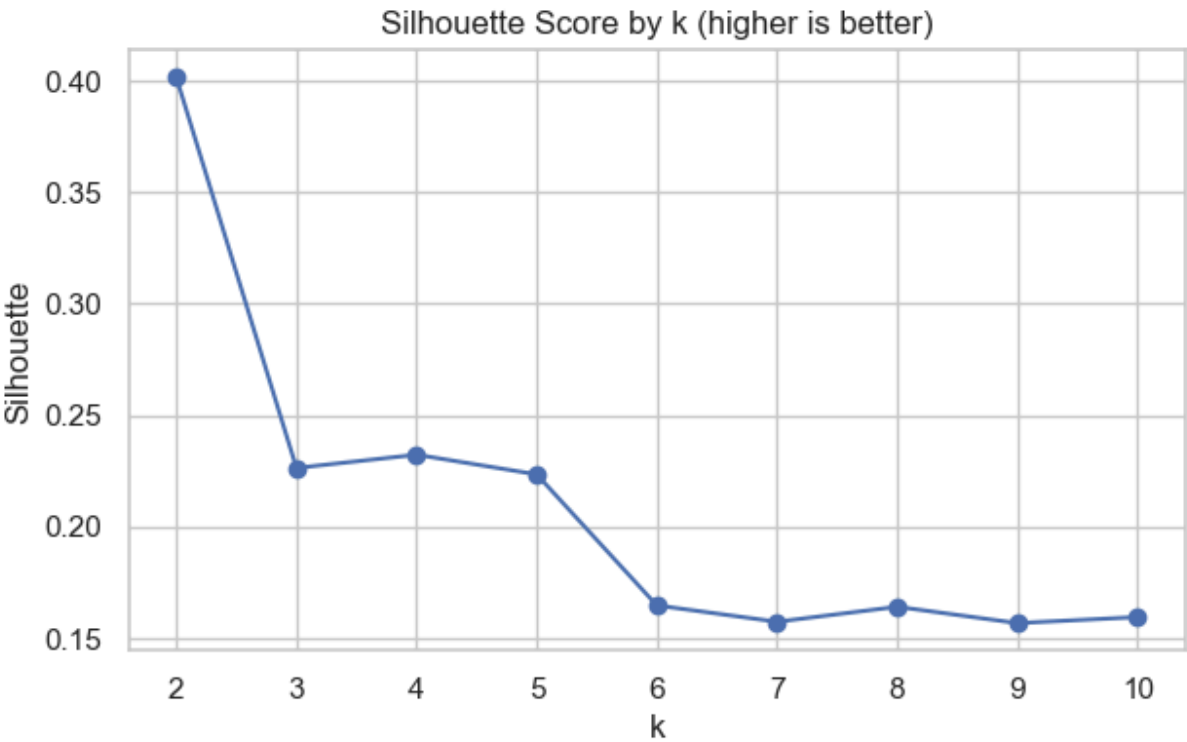
```
inertias.append(km.inertia_)

plt.figure(figsize=(7,4))
plt.plot(Ks, inertias, marker='o')
plt.title("Elbow Method for k")
plt.xlabel("k"); plt.ylabel("Within-Cluster SS (inertia)")
plt.xticks(Ks); plt.grid(True); plt.show()

# Silhouette
Ks2 = range(2, 11)
sil_scores = []
for k in Ks2:
    labels = KMeans(n_clusters=k, n_init=25, random_state=42).fit_predict(X)
    sil_scores.append(silhouette_score(X_scaled, labels))

plt.figure(figsize=(7,4))
plt.plot(Ks2, sil_scores, marker='o')
plt.title("Silhouette Score by k (higher is better)")
plt.xlabel("k"); plt.ylabel("Silhouette")
plt.xticks(Ks2); plt.grid(True); plt.show()
```





```
In [6]: k = 4 # based on the elbow plot
kmeans = KMeans(n_clusters=k, n_init=50, random_state=42)
labels = kmeans.fit_predict(X_scaled)

df_family_segments['cluster'] = labels

# Cluster means (ORIGINAL UNITS)
cluster_means = df_family_segments.groupby('cluster')[numeric_cols].mean().round(2)
display(cluster_means)

# Categorical proportions by cluster
cluster_props = df_family_segments.groupby('cluster')[cat_cols].mean().round(2)
display(cluster_props.rename(columns={'domestic': '% domestic', 'dis_plus_suburb': '% dis_plus_suburb'}
```

	est_net_worth	est_inc	number_visits	number_children	oldest_child_age	oldest_child_age_o
cluster						
0	356300.0	294681.0	13.0	2.0	12.0	
1	55817.0	42112.0	2.0	2.0	12.0	
2	671408.0	536203.0	2.0	2.0	12.0	
3	4107011.0	2592323.0	3.0	2.0	12.0	

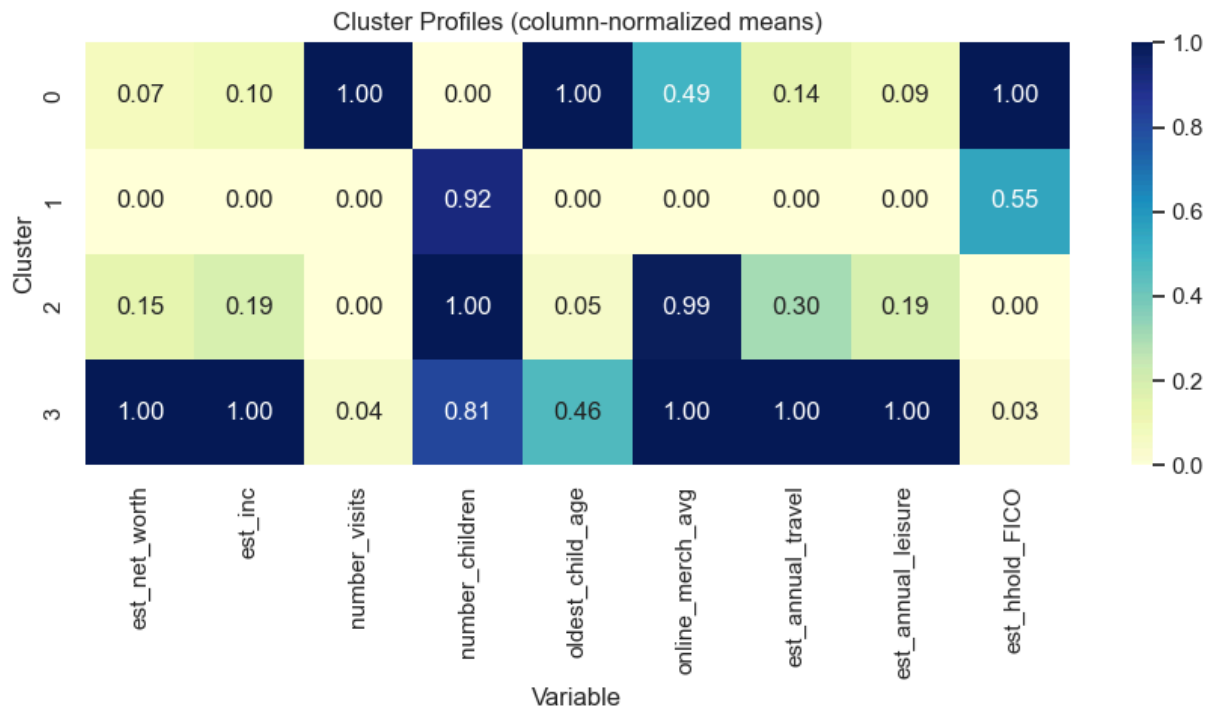


	% domestic	% Disney+ sub
cluster		
0	0.744	0.665
1	0.721	0.474
2	0.729	0.813
3	0.712	0.809

In [8]: *# Vizualizations*

```
palette = sns.color_palette("Set2", n_colors=k)

# A) Heat map of normalized cluster means (pattern view)
cm = df_family_segments.groupby('cluster')[numeric_cols].mean()
cm_norm = (cm - cm.min()) / (cm.max() - cm.min()) # 0-1 by column
plt.figure(figsize=(9,5))
sns.heatmap(cm_norm, annot=True, fmt=".2f", cmap="YlGnBu")
plt.title("Cluster Profiles (column-normalized means)")
plt.xlabel("Variable"); plt.ylabel("Cluster")
plt.tight_layout(); plt.show()
```



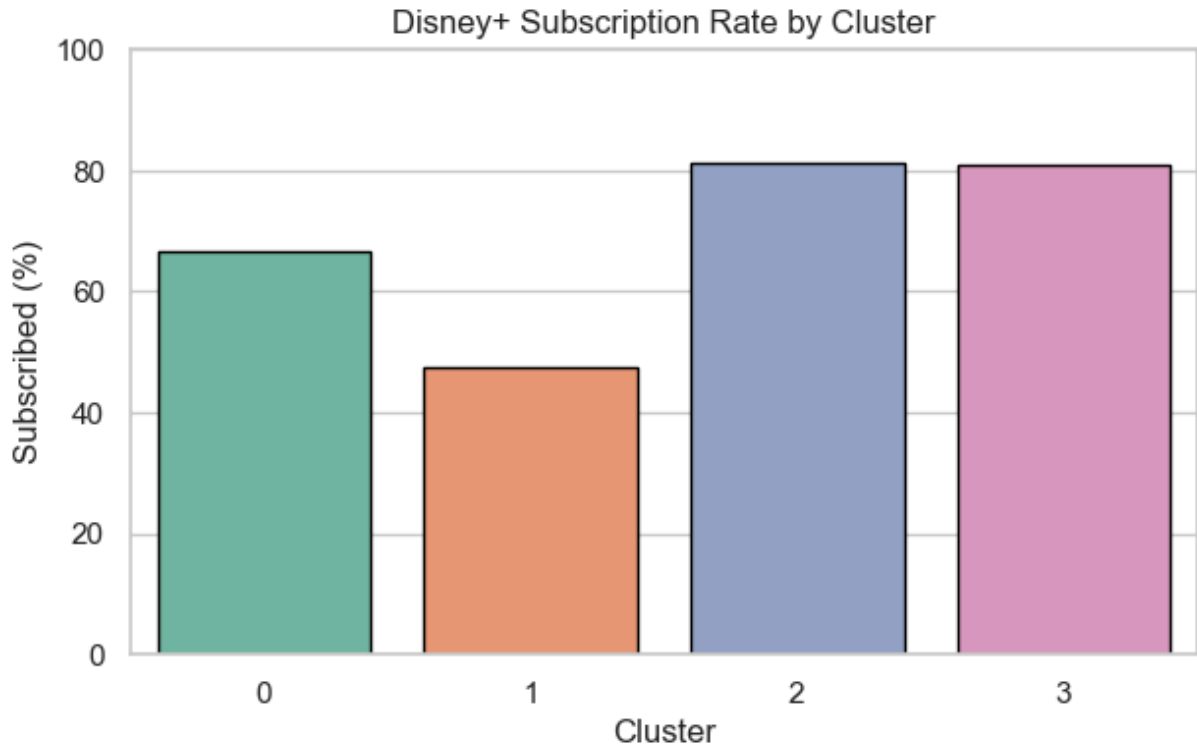
In [9]: *# B) Disney+ subscription rate (%) by cluster*

```
sub_rate = (df_family_segments.groupby('cluster')['dis_plus_sub'].mean()*100)
plt.figure(figsize=(7,4))
sns.barplot(data=sub_rate, x='cluster', y='Subscribed (%)', palette=palette,
plt.title('Disney+ Subscription Rate by Cluster'); plt.ylim(0,100); plt.xlabel('cluster')
```

```
/var/folders/xq/hz_jsm8n7916kt1g6zzwf7cm0000gn/T/ipykernel_82629/1456287839.py:4: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=sub_rate, x='cluster', y='Subscribed (%)', palette=palette, edgecolor='black')
```

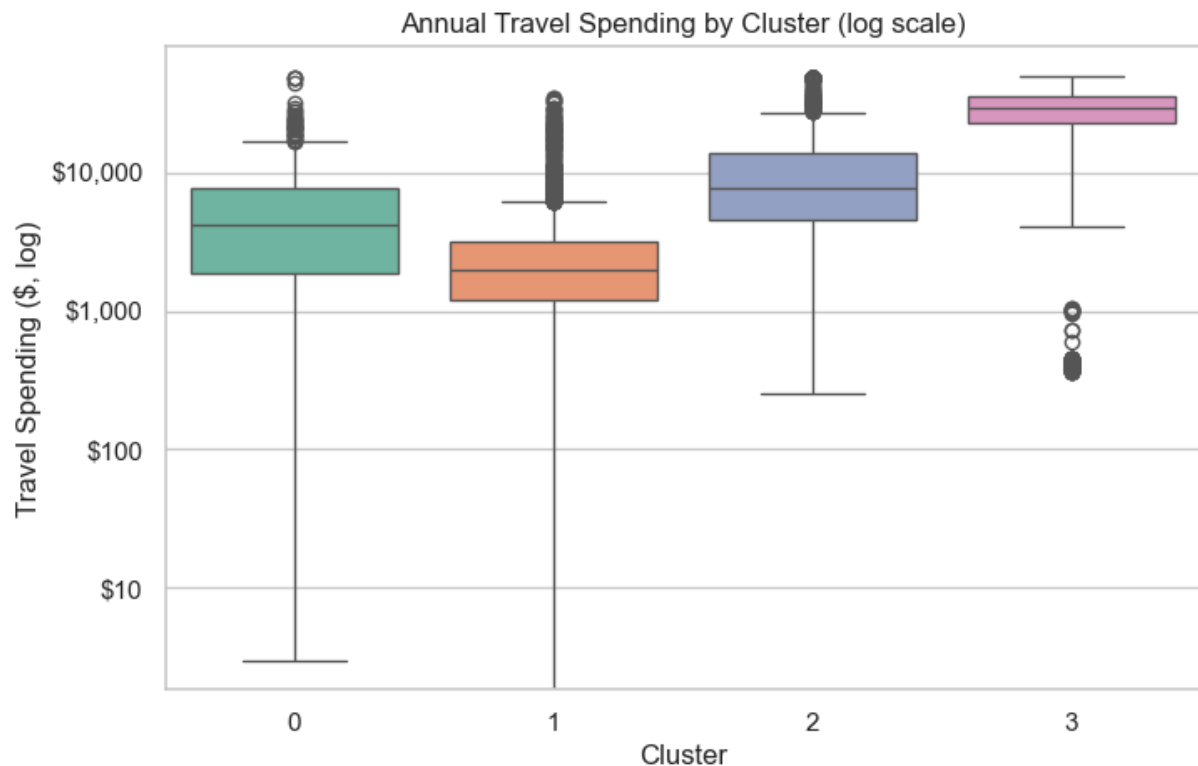


```
In [10]: # C) Annual travel spending by cluster (skewed → log scale)
plt.figure(figsize=(8,5))
ax = sns.boxplot(data=df_family_segments, x='cluster', y='est_annual_travel')
ax.set_yscale('log'); ax.yaxis.set_major_formatter(fmt_dollar)
plt.title('Annual Travel Spending by Cluster (log scale)')
plt.xlabel('Cluster'); plt.ylabel('Travel Spending ($, log)'); plt.show()
```

```
/var/folders/xq/hz_jsm8n7916kt1g6zzwf7cm0000gn/T/ipykernel_82629/1554697860.py:3: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

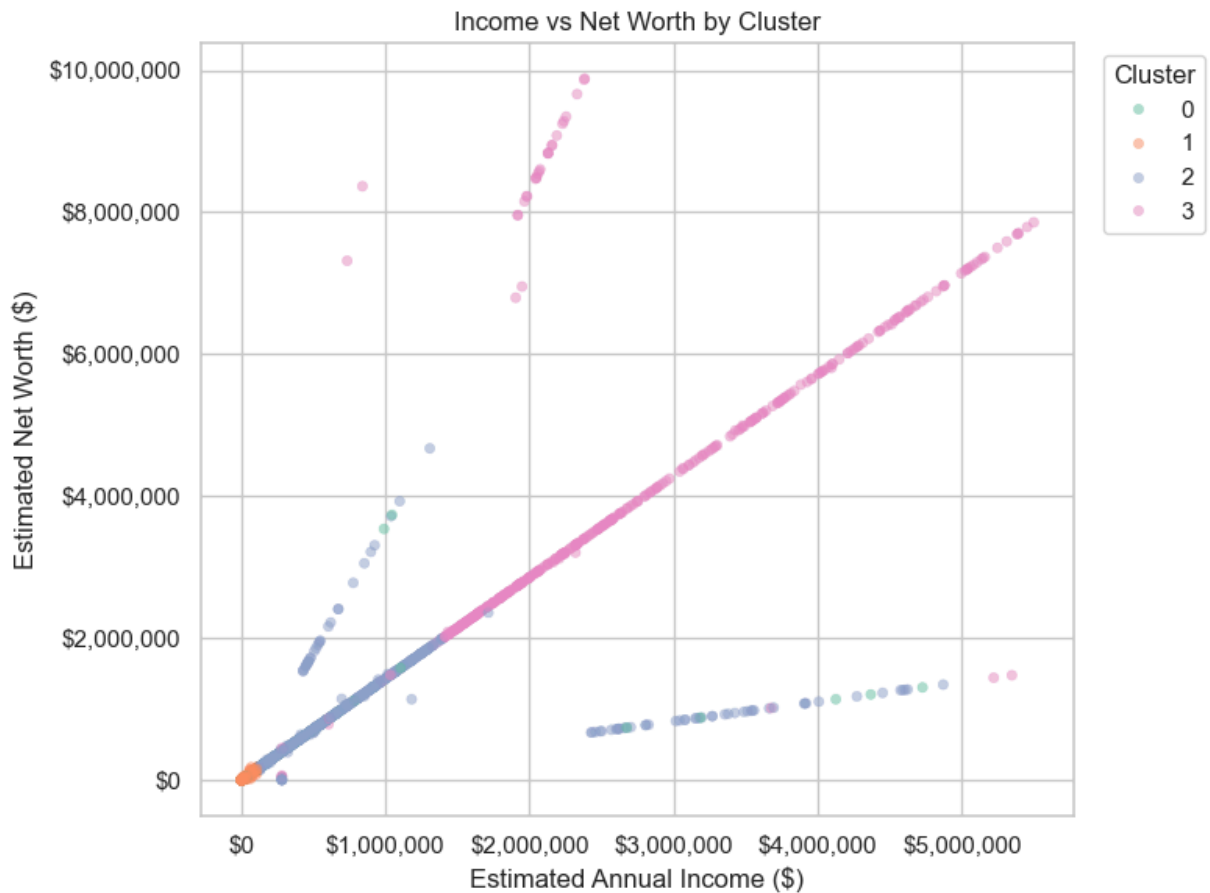
```
ax = sns.boxplot(data=df_family_segments, x='cluster', y='est_annual_travel', palette=palette, showfliers=True)
```



```
In [11]: # D) Scatter: Park visits vs Online Merch spend (trim big tails)
plt.figure(figsize=(8,6))
sns.scatterplot(
    data=df_family_segments[df_family_segments['online_merch_avg']<=5000],
    x='number_visits', y='online_merch_avg',
    hue='cluster', palette=palette, alpha=0.6, s=22, edgecolor=None
)
plt.title('Park Visits vs Online Merchandise Spend by Cluster')
plt.xlabel('Number of Visits'); plt.ylabel('Annual Online Merch Spend ($)')
plt.gca().yaxis.set_major_formatter(fmt_dollar)
plt.legend(title='Cluster', bbox_to_anchor=(1.02,1), loc='upper left')
plt.tight_layout(); plt.show()
```



```
In [12]: # E) Scatter: Income vs Net Worth (subsample for readability)
sample = df_family_segments.sample(min(4000, len(df_family_segments)), random_state=42)
plt.figure(figsize=(8,6))
sns.scatterplot(
    data=sample, x='est_inc', y='est_net_worth',
    hue='cluster', palette=palette, alpha=0.5, s=20, edgecolor=None
)
plt.title('Income vs Net Worth by Cluster')
plt.xlabel('Estimated Annual Income ($)'); plt.ylabel('Estimated Net Worth ($)')
plt.gca().xaxis.set_major_formatter(fmt_dollar)
plt.gca().yaxis.set_major_formatter(fmt_dollar)
plt.legend(title='Cluster', bbox_to_anchor=(1.02,1), loc='upper left')
plt.tight_layout(); plt.show()
```



What the Vizualizations show about the model

1. **Heatmap of Normalized Cluster Means** This chart shows the average values for each numeric variable in each cluster, scaled from 0 to 1 for easy comparison. Darker colors mean higher relative values within that variable. It helps us see which clusters stand out for things like income, visits, or merchandise spending.
2. **Disney+ Subscription Rate by Cluster** This bar chart shows the percentage of households in each cluster that subscribe to Disney+. Higher bars mean more Disney+ subscribers in that group. It can help Disney target clusters that are already engaged with its brand.
3. **Annual Travel Spending by Cluster (Log Scale)** This boxplot compares yearly travel spending across clusters, using a log scale to handle extreme values. The middle line in each box is the median spending, and the spread shows variation within the cluster. It helps identify which clusters are more likely to spend heavily on travel.
4. **Park Visits vs. Online Merchandise Spend by Cluster** This scatterplot shows the relationship between park visits and online merchandise spending. Each dot is a household, colored by cluster, so we can see which clusters tend to visit more and spend more online. It highlights behavioral differences between frequent visitors and online shoppers.

5. **Income vs. Net Worth by Cluster** This scatterplot shows each household's estimated annual income and net worth, colored by cluster. It reveals the financial capacity of each group, with some clusters having both high income and net worth, and others having lower levels. This helps identify high-value customer segments for premium offerings.

Choosing the Number of Clusters

We used the **Elbow Method** to decide how many clusters to use. By plotting the WCSS (within-cluster sum of squares) for different numbers of clusters, we saw the curve bend noticeably at **k = 4**. This means 4 clusters gave us a good balance between accuracy and simplicity, avoiding both underfitting and overfitting.

Cluster 0: High Spenders Description:

Families in this group have very high income and net worth. They travel frequently, spend a lot on Disney merchandise, and often book premium vacations. They are already highly engaged with the brand and value convenience, exclusivity, and memorable experiences.

Targeting:

Promote VIP tours, luxury resort stays, early access to attractions, and limited-edition merchandise. Use personalized marketing and loyalty perks to keep them coming back.

Cluster 1: Budget Families Description:

These families have lower income and net worth, make fewer park visits, and spend less on travel and merchandise. They may enjoy Disney but are more sensitive to price and look for affordable options. They are more likely to respond to savings and local promotions.

Targeting:

Offer discounted tickets, seasonal promotions, and local resident deals. Use value-focused messaging, family bundles, and flexible payment options to make visits more accessible.

Cluster 2: Park Lovers Description:

This group has moderate income and net worth but visits the parks often. They spend significantly on travel and merchandise relative to their means and have a strong emotional connection to the Disney brand. They are active participants in park activities and return multiple times a year.

Targeting:

Highlight annual passes, multi-visit discounts, and exclusive event access. Bundle park tickets with merchandise offers to reward loyalty and encourage spending during visits.

Cluster 3: Wealthy Shoppers Description:

Households in this cluster have the highest income and net worth of all groups. They spend heavily on travel, leisure, and online Disney shopping, even if they visit the parks

less often. They appreciate quality and are willing to pay for premium goods and experiences.

Targeting:

Showcase luxury travel packages, exclusive collector's merchandise, and private event invitations. Use targeted advertising for high-end experiences and personalized concierge services.

IV. Conjoint Analysis

In [2]: `import pandas as pd`

```
df= pd.read_csv("../AD 652 Final Project/hotel_amenities.csv")
print(df.head())
print(df.dtypes)
```

```
WiFi_Network breakfast parking gym flex_check shuttle_bus air_pure jacuzz
i \
0      Basic      NaN  Valet  NaN      No      No      No      N
0
1      Basic      NaN  Valet  NaN      No      No      No      N
0
2      Basic      NaN  Valet  NaN      No      No      No      N
0
3      Basic      NaN  Valet  NaN      No      No      No      N
0
4      Basic      NaN  Valet  NaN      No      No      No      N
0
```

```
VIP_shop pool_temp avg_rating
0      No      76      4.57
1      No      80      7.60
2      No      84      5.66
3      Yes     76      2.80
4      Yes     80      4.56
```

```
WiFi_Network object
breakfast object
parking object
gym object
flex_check object
shuttle_bus object
air_pure object
jacuzzi object
VIP_shop object
pool_temp int64
avg_rating float64
dtype: object
```

In [11]: `#'None' was auto changed to NaN, fix it or else during dummification, it will`
`df['breakfast']= df['breakfast'].fillna('None')`
`df['gym']= df['gym'].fillna('None')`

```

cat_cols= ['WiFi_Network', 'breakfast', 'parking', 'gym', 'flex_check', 'shut
df[cat_cols]= df[cat_cols].astype('category')

y= df['avg_rating']
x = df.drop(columns=['avg_rating'])

#dummify cat vars
x_dummies= pd.get_dummies(x, drop_first=True)
print(x_dummies.head())

```

	WiFi_Network_Best	in Class	WiFi_Network_Strong	breakfast_Full	Buffet	\
0		False	False		False	
1		False	False		False	
2		False	False		False	
3		False	False		False	
4		False	False		False	

	breakfast_None	parking_Valet	gym_Basic	gym_None	gym_Super	\
0	True	True	False	True	False	
1	True	True	False	True	False	
2	True	True	False	True	False	
3	True	True	False	True	False	
4	True	True	False	True	False	

	flex_check_Yes	shuttle_bus_Yes	air_pure_Yes	jacuzzi_Yes	VIP_shop_Yes	\
0	False	False	False	False	False	
1	False	False	False	False	False	
2	False	False	False	False	False	
3	False	False	False	False	True	
4	False	False	False	False	True	

	pool_temp_80	pool_temp_84
0	False	False
1	True	False
2	False	True
3	False	False
4	True	False

```

In [12]: import statsmodels.api as sm
x_dummies = x_dummies.astype(float)
#intercept
x_dummies= sm.add_constant(x_dummies)
#fit
model= sm.OLS(y, x_dummies).fit()

print(model.summary())

```

OLS Regression Results

```

=====
==
Dep. Variable:          avg_rating    R-squared:                0.2
77
Model:                  OLS          Adj. R-squared:           0.2
75
Method:                 Least Squares    F-statistic:              17
5.7
Date:                   Sun, 10 Aug 2025    Prob (F-statistic):       0.
00
Time:                   19:28:19          Log-Likelihood:           -1217
6.
No. Observations:      6912             AIC:                      2.438e+
04
Df Residuals:          6896             BIC:                      2.449e+
04
Df Model:               15
Covariance Type:       nonrobust
=====
=====

```

```

=====
=====
                                coef    std err          t      P>|t|
-----
[0.025    0.975]
-----
const                        5.5280    0.068      81.468    0.000
5.395    5.661
WiFi_Network_Best in Class   1.7268    0.042     41.558    0.000
1.645    1.808
WiFi_Network_Strong         1.1877    0.042     28.583    0.000
1.106    1.269
breakfast_Full Buffet       0.5010    0.042     12.057    0.000
0.420    0.582
breakfast_None              -0.2259    0.042     -5.437    0.000
-0.307   -0.144
parking_Valet               0.0937    0.034      2.761    0.006
0.027    0.160
gym_Basic                   -0.1463    0.048     -3.049    0.002
-0.240   -0.052
gym_None                    -0.1685    0.048     -3.512    0.000
-0.263   -0.074
gym_Super                   0.0444    0.048      0.925    0.355
-0.050    0.138
flex_check_Yes              0.4782    0.034     14.096    0.000
0.412    0.545
shuttle_bus_Yes             0.4199    0.034     12.378    0.000
0.353    0.486
air_pure_Yes                0.0753    0.034      2.218    0.027
0.009    0.142
jacuzzi_Yes                 0.1839    0.034      5.421    0.000
0.117    0.250
VIP_shop_Yes                0.2179    0.034      6.423    0.000
0.151    0.284
pool_temp_80                0.0747    0.042      1.799    0.072
-0.007    0.156
pool_temp_84                0.2638    0.042      6.349    0.000

```

```

0.182      0.345
=====
==
Omnibus:      153.138   Durbin-Watson:      1.9
48
Prob(Omnibus):      0.000   Jarque-Bera (JB):      162.9
51
Skew:      -0.370   Prob(JB):      4.13e-
36
Kurtosis:      3.130   Cond. No.      9.
43
=====
==

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

In [13]: costs= pd.read_csv('amenity_costs.csv')
          print(costs)
          print(costs.dtypes)

```

	Amenity	Level \
0	WiFi_Network	Basic
1	WiFi_Network	Strong
2	WiFi_Network	Best in Class
3	breakfast	NaN
4	breakfast	Continental
5	breakfast	Full Buffet
6	parking	Valet
7	parking	Open Lot
8	gym	NaN
9	gym	Basic
10	gym	Advanced
11	gym	Super
12	flex_check	No
13	flex_check	Yes
14	shuttle_bus	No
15	shuttle_bus	Yes
16	air_pure	No
17	air_pure	Yes
18	jacuzzi	No
19	jacuzzi	Yes
20	VIP_shop	No
21	VIP_shop	Yes
22	pool temp	76
23	pool temp	80
24	pool temp	84

	Estimated Incremental Cost,\nPer Visitor/Per Night
0	11.75
1	16.25
2	19.15
3	0.00
4	13.25
5	22.45
6	60.00
7	15.00
8	0.00
9	10.00
10	35.00
11	65.00
12	0.00
13	12.00
14	0.00
15	75.00
16	0.00
17	12.85
18	0.00
19	40.00
20	0.00
21	12.00
22	15.00
23	35.00
24	45.00

Amenity

object

Level

object

Estimated Incremental Cost,\nPer Visitor/Per Night float64
dtype: object

```
In [14]: #match col names to model so that we can merge later
costs['Level'] = costs['Level'].fillna('None')
costs['Amenity'] = costs['Amenity'].replace({'pool temp': 'pool_temp'})
costs['dummy_name'] = costs['Amenity'] + '_' + costs['Level'].astype(str)

print(costs[['dummy_name']])
```

```

            dummy_name
0      WiFi_Network_Basic
1      WiFi_Network_Strong
2  WiFi_Network_Best in Class
3      breakfast_None
4      breakfast_Continental
5      breakfast_Full Buffet
6      parking_Valet
7      parking_Open Lot
8              gym_None
9              gym_Basic
10             gym_Advanced
11             gym_Super
12      flex_check_No
13      flex_check_Yes
14      shuttle_bus_No
15      shuttle_bus_Yes
16      air_pure_No
17      air_pure_Yes
18      jacuzzi_No
19      jacuzzi_Yes
20      VIP_shop_No
21      VIP_shop_Yes
22      pool_temp_76
23      pool_temp_80
24      pool_temp_84
```

```
In [15]: #create df for model parameters
coeff_df = pd.DataFrame(model.params).reset_index()
coeff_df.columns = ['dummy_name', 'coeff']

#merge w the costs
merged = pd.merge(costs, coeff_df, on='dummy_name', how='left')
merged = merged.dropna(subset=['coeff'])

print(merged)
```

	Amenity	Level \
1	WiFi_Network	Strong
2	WiFi_Network	Best in Class
3	breakfast	None
5	breakfast	Full Buffet
6	parking	Valet
8	gym	None
9	gym	Basic
11	gym	Super
13	flex_check	Yes
15	shuttle_bus	Yes
17	air_pure	Yes
19	jacuzzi	Yes
21	VIP_shop	Yes
23	pool_temp	80
24	pool_temp	84

	Estimated Incremental Cost,\nPer Visitor/Per Night \
1	16.25
2	19.15
3	0.00
5	22.45
6	60.00
8	0.00
9	10.00
11	65.00
13	12.00
15	75.00
17	12.85
19	40.00
21	12.00
23	35.00
24	45.00

	dummy_name	coeff
1	WiFi_Network_Strong	1.187700
2	WiFi_Network_Best in Class	1.726814
3	breakfast_None	-0.225924
5	breakfast_Full Buffet	0.500998
6	parking_Valet	0.093678
8	gym_None	-0.168484
9	gym_Basic	-0.146314
11	gym_Super	0.044363
13	flex_check_Yes	0.478220
15	shuttle_bus_Yes	0.419939
17	air_pure_Yes	0.075258
19	jacuzzi_Yes	0.183909
21	VIP_shop_Yes	0.217925
23	pool_temp_80	0.074744
24	pool_temp_84	0.263806

```
In [16]: merged['cost']= merged['Estimated Incremental Cost,\nPer Visitor/Per Night']
merged['utility_per_dollar'] = merged['coeff']/merged['cost'].replace(0, 0.000001)

merged_sorted= merged.sort_values(by='utility_per_dollar', ascending=False)
print(merged_sorted)
```

	Amenity	Level \
2	WiFi_Network	Best in Class
1	WiFi_Network	Strong
13	flex_check	Yes
5	breakfast	Full Buffet
21	VIP_shop	Yes
24	pool_temp	84
17	air_pure	Yes
15	shuttle_bus	Yes
19	jacuzzi	Yes
23	pool_temp	80
6	parking	Valet
11	gym	Super
9	gym	Basic
8	gym	None
3	breakfast	None

	Estimated Incremental Cost,\nPer Visitor/Per Night \
2	19.15
1	16.25
13	12.00
5	22.45
21	12.00
24	45.00
17	12.85
15	75.00
19	40.00
23	35.00
6	60.00
11	65.00
9	10.00
8	0.00
3	0.00

	dummy_name	coeff	cost	utility_per_dollar
2	WiFi_Network_Best in Class	1.726814	19.15	0.090173
1	WiFi_Network_Strong	1.187700	16.25	0.073089
13	flex_check_Yes	0.478220	12.00	0.039852
5	breakfast_Full Buffet	0.500998	22.45	0.022316
21	VIP_shop_Yes	0.217925	12.00	0.018160
24	pool_temp_84	0.263806	45.00	0.005862
17	air_pure_Yes	0.075258	12.85	0.005857
15	shuttle_bus_Yes	0.419939	75.00	0.005599
19	jacuzzi_Yes	0.183909	40.00	0.004598
23	pool_temp_80	0.074744	35.00	0.002136
6	parking_Valet	0.093678	60.00	0.001561
11	gym_Super	0.044363	65.00	0.000683
9	gym_Basic	-0.146314	10.00	-0.014631
8	gym_None	-0.168484	0.00	-1684.837963
3	breakfast_None	-0.225924	0.00	-2259.244792

```
In [17]: selected= []
total_cost= 0
amenities_selected= set()
```

#need to make sure only one level per amenity is selected

```

for _, row in merged_sorted.iterrows():
    if row['Amenity'] not in amenities_selected:
        if total_cost + row['cost'] <= 250:
            selected.append(row)
            total_cost += row['cost']
            amenities_selected.add(row['Amenity'])

selected_df= pd.DataFrame(selected)
total_predicted_rating= selected_df['coeff'].sum() + model.params['const']

print(selected_df[['Amenity', 'Level', 'cost', 'coeff']])
print(f"Total Cost: ${total_cost}")

print(f"Total Rating Improvement from Amenities Only: {selected_df['coeff'].sum()}")
print(f"Total Overall Predicted Rating (With intercept): {total_predicted_rating}")

```

	Amenity	Level	cost	coeff
2	WiFi_Network	Best in Class	19.15	1.726814
13	flex_check	Yes	12.00	0.478220
5	breakfast	Full Buffet	22.45	0.500998
21	VIP_shop	Yes	12.00	0.217925
24	pool_temp	84	45.00	0.263806
17	air_pure	Yes	12.85	0.075258
15	shuttle_bus	Yes	75.00	0.419939
19	jacuzzi	Yes	40.00	0.183909
9	gym	Basic	10.00	-0.146314

Total Cost: \$248.45
Total Rating Improvement from Amenities Only: 3.721
Total Overall Predicted Rating (With intercept): 9.249

Conjoint Analysis

The analysis used two datasets: survey ratings for different hotel amenity options and the estimated nightly cost of providing each one. "None" options were kept as actual values so they would be treated as valid categories in the model.

Each amenity was converted into dummy variables, with one category as the baseline. An OLS regression estimated how much each amenity level changed guest ratings compared to that baseline. The resulting coefficients were matched to the cost data, and a "utility per dollar" score was calculated for each option.

Amenities were ranked by this score, and the highest-ranked ones were chosen until the **\$250** per room, per night budget limit was reached. Only one level per amenity type was included.

The final combination of amenities, totaling **\$248.45**, is as follows:

- Best in Class Wi-Fi
- Flexible check-in
- Full buffet breakfast
- VIP shopping access
- Heated pool (84°F)

- Air purification
- Shuttle bus service
- Jacuzzi
- Basic gym

This set of amenities is expected to raise guest ratings by **3.72** points over the basic offering, giving the hotel an overall predicted rating of **9.25**. The selection covers the essentials most valued by guests (reliable Wi-Fi, flexible check-in, and a full buffet breakfast), which addresses convenience and comfort needs.

It also includes additional amenities that are considered high-end such as heated pool, jacuzzi, and VIP shopping, which means this set also appeals to leisure travelers seeking memorable vacation experiences. Air purification and a shuttle bus provide practical benefits that improve overall satisfaction without adding large costs.

The final selection ensures the combined cost of amenities are still within the **\$250** limit per room, while still matching what survey respondents rated most highly. This gives the hotel a strong chance of attracting and satisfying guests in Hawaii.

Recommendation

Disney should consider including these amenities to provide a hotel experience that guests will rate highly while controlling costs:

- Prioritize the amenities with high rating improvement per dollar spent, such as Wi-Fi with "Best in Class", flexible check-in which likely reduces stress for guests who try to arrive on time, and full buffet breakfast.
- Certain popular amenities have higher costs, but should still be included as it has a positive impact on guest ratings, like shuttle service and heated pool.
- Some amenities being offered should possibly be re-evaluated due to their negative or low contributions to guest satisfaction, such as the basic gym.
- This model should help price rooms competitively and design the hotel to maximize guest satisfaction within the \$250 per room budget.

V. Forecasting

Disney Net Income Forecast for 2025

```
In [3]: from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

disneyi = pd.read_csv("../AD 652 Final Project/disney_net_income.csv")
disneyi.head()
```

Out [3]:

	year	net_income
0	2009	3307
1	2010	3963
2	2011	4807
3	2012	5682
4	2013	6136

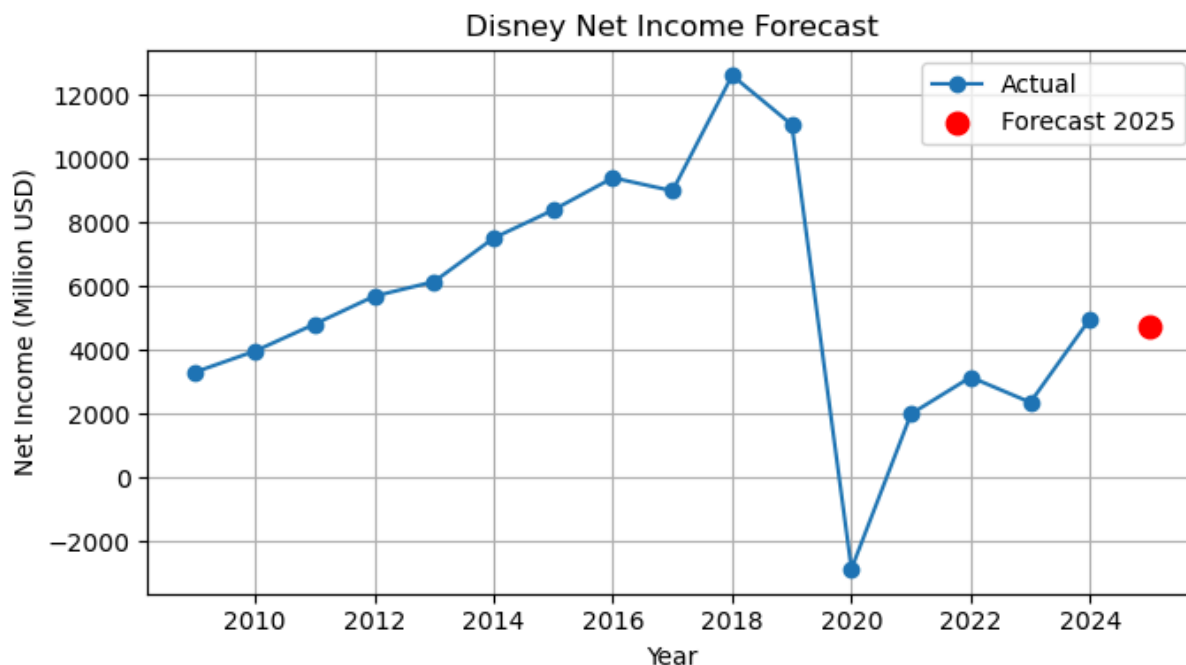
```
In [4]: # Prepare features (X) and target (y)
X = disneyi[["year"]]
y = disneyi["net_income"]

# Fit linear regression model
model = LinearRegression()
model.fit(X, y)

# Forecast for 2025
forecast_year = 2025
forecast_value = model.predict([[forecast_year]])[0]
print(f"Forecasted Net Income for {forecast_year}: {forecast_value:,.0f} mil

# Plot actual vs. forecast
plt.figure(figsize=(7,4))
plt.plot(disneyi["year"], disneyi["net_income"], marker="o", label="Actual")
plt.scatter(forecast_year, forecast_value, color="red", s=80, label=f"Forecast")
plt.title("Disney Net Income Forecast")
plt.xlabel("Year")
plt.ylabel("Net Income (Million USD)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
/opt/anaconda3/lib/python3.13/site-packages/sklearn/utils/validation.py:273
9: UserWarning: X does not have valid feature names, but LinearRegression was
fitted with feature names
warnings.warn(
Forecasted Net Income for 2025: 4,708 million USD
```



```
In [5]: print(f"Forecasted Net Income for {forecast_year}: {forecast_value:,.0f} mil
```

Forecasted Net Income for 2025: 4,708 million USD

Forecast Results

Based on Disney's net income from 2009 to 2024, the forecast for 2025 is about 4.71 billion USD. From 2009 to 2018, Disney's profits grew steadily and reached over 12 billion USD. In 2020, profits dropped sharply because of COVID-19, which closed theme parks, stopped movie releases, and reduced cruise trips. Profits have improved since then, but the recovery has been uneven due to changes in customer habits, more competition in streaming, and a slower global economy. The 2025 forecast shows a stable profit level, but still far below the best years before the pandemic, meaning Disney may need more time and strategy to fully recover.

VI. Classification: Logistic Regression

```
In [16]: import pandas as pd, numpy as np
```

```
In [19]: df_cruise_returns = pd.read_csv("../AD 652 Final Project/cruise_returns.
df_cruise_returns.head()
```

Out [19]:

	hhold_ID	discount_original	coupon_received	cabin_type	est_inc	season_of
0	3598	1	1	Verandah	19280.985	\
1	155	1	0	Concierge	825719.960	\$
2	353	1	0	Oceanview	3755246.459	
3	1474	1	1	Oceanview	96831.550	\
4	3272	1	0	Verandah	11027.249	\$

In [20]: `df_cruise_returners.isnull().sum()`

Out [20]:

hhold_ID	0
discount_original	0
coupon_received	0
cabin_type	0
est_inc	0
season_of_year	0
ticket_spend	0
cruise_theme	1250
incidental_spending	0
park_package	0
embark_port	0
res_to_port	0
number_children	0
oldest_child_age	370
dis_plus_sub	0
return_1065	0
dtype:	int64

In [21]:

```
# Cleaning the dataset

# Remove ID column
if 'householdID' in df_cruise_returners.columns:
    df_cruise_returners.drop(columns=['householdID'], inplace=True)

# Check missing values
missing_counts = df_cruise_returners.isnull().sum()
print("Missing values per column:\n", missing_counts)

# Drop columns with >30% missing
threshold = 0.3 * len(df_cruise_returners)
cols_to_drop = missing_counts[missing_counts > threshold].index
df_cruise_returners.drop(columns=cols_to_drop, inplace=True)

# Fill numeric NaNs with median
num_cols = df_cruise_returners.select_dtypes(include=['number']).columns
for col in num_cols:
    df_cruise_returners[col].fillna(df_cruise_returners[col].median(), inplace=True)
```

```

# Fill categorical NaNs with mode
cat_cols = df_cruise_returners.select_dtypes(exclude=['number']).columns
for col in cat_cols:
    df_cruise_returners[col].fillna(df_cruise_returners[col].mode()[0], inplace=True)

# Drop constant columns
constant_cols = [col for col in df_cruise_returners.columns if df_cruise_returners[col].nunique() == 1]
df_cruise_returners.drop(columns=constant_cols, inplace=True)

print("Final columns after cleaning:", df_cruise_returners.columns.tolist())
df_cruise_returners.head()

```

Missing values per column:

hhold_ID	0
discount_original	0
coupon_received	0
cabin_type	0
est_inc	0
season_of_year	0
ticket_spend	0
cruise_theme	1250
incidental_spending	0
park_package	0
embark_port	0
res_to_port	0
number_children	0
oldest_child_age	370
dis_plus_sub	0
return_1065	0

dtype: int64

Final columns after cleaning: ['hhold_ID', 'discount_original', 'coupon_received', 'cabin_type', 'est_inc', 'season_of_year', 'ticket_spend', 'cruise_theme', 'incidental_spending', 'park_package', 'embark_port', 'res_to_port', 'number_children', 'oldest_child_age', 'dis_plus_sub', 'return_1065']

```
/var/folders/xq/hz_jsm8n7916kt1g6zzwf7cm0000gn/T/ipykernel_82629/4092670643.py:19: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df_cruise_returners[col].fillna(df_cruise_returners[col].median(), inplace=True)
/var/folders/xq/hz_jsm8n7916kt1g6zzwf7cm0000gn/T/ipykernel_82629/4092670643.py:24: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df_cruise_returners[col].fillna(df_cruise_returners[col].mode()[0], inplace=True)
```

```
Out[21]:
```

	hhold_ID	discount_original	coupon_received	cabin_type	est_inc	season_of
0	3598	1	1	Verandah	19280.985	\
1	155	1	0	Concierge	825719.960	\$
2	353	1	0	Oceanview	3755246.459	
3	1474	1	1	Oceanview	96831.550	\
4	3272	1	0	Verandah	11027.249	\$

```
In [22]: # Step 1: Check Class Balance
df_cruise_returners['return_1065'].value_counts()
df_cruise_returners['return_1065'].value_counts(normalize=True)
```

```
Out[22]: return_1065
0      0.52
1      0.48
Name: proportion, dtype: float64
```

```
In [23]: # Step 2: Train/Test Split

from sklearn.model_selection import train_test_split
```

```

X = df_cruise_returners.drop(columns=['return_1065'])
y = df_cruise_returners['return_1065']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=1512
)

# Step 3: Explore Training Data

X_train.describe()

```

Out[23]:

	hhold_ID	discount_original	coupon_received	est_inc	ticket_spend
count	4500.000000	4500.000000	4500.000000	4.500000e+03	4500.000000
mean	3729.107333	0.386444	0.255778	5.898086e+05	2315.758444
std	2171.292727	0.486989	0.436346	1.446464e+06	570.481007
min	1.000000	0.000000	0.000000	9.070000e-01	1102.000000
25%	1852.250000	0.000000	0.000000	7.562690e+03	1888.750000
50%	3708.500000	0.000000	0.000000	5.157657e+04	2305.500000
75%	5630.500000	1.000000	1.000000	3.540219e+05	2724.250000
max	7497.000000	1.000000	1.000000	9.938626e+06	3600.000000

In [24]:

```

# Step 4: Check Correlations
numeric_cols = X_train.select_dtypes(include=['number']).columns
X_train[numeric_cols].corr()

```

Out [24]:

	hhold_ID	discount_original	coupon_received	est_inc	ticket_
hhold_ID	1.000000	-0.009646	0.002753	0.012282	0.0
discount_original	-0.009646	1.000000	-0.008156	0.001369	-0.0
coupon_received	0.002753	-0.008156	1.000000	0.006459	-0.0
est_inc	0.012282	0.001369	0.006459	1.000000	0.0
ticket_spend	0.009950	-0.025417	-0.005612	0.013367	1.0
incidental_spending	0.012967	-0.035754	0.009117	0.647024	0.0
park_package	0.002550	-0.023896	-0.002219	0.014071	0.0
res_to_port	0.001789	0.017741	0.012729	-0.005764	0.0
number_children	0.007785	-0.013320	0.008009	-0.038997	-0.0
oldest_child_age	-0.016002	-0.016892	-0.002865	-0.003130	-0.0
dis_plus_sub	-0.005033	0.036976	0.019274	0.018245	0.0

```

In [25]: # Step 5: Dummy Code Categorical Variables
X_train = pd.get_dummies(X_train, drop_first=True)
X_test = pd.get_dummies(X_test, drop_first=True)

# Align train and test columns
X_test = X_test.reindex(columns=X_train.columns, fill_value=0)

```

```

In [28]: # Convert everything to float for statsmodels
X_train = X_train.astype(float)
X_test = X_test.astype(float)

# ITERATION 1: Logistic Regression (All Predictors)
import statsmodels.api as sm
logit_model1 = sm.Logit(y_train, sm.add_constant(X_train))
result1 = logit_model1.fit()
print(result1.summary())

```

Optimization terminated successfully.

Current function value: 0.648063

Iterations 5

Logit Regression Results

```
=====
==
Dep. Variable:          return_1065    No. Observations:          45
00
Model:                  Logit          Df Residuals:              44
73
Method:                 MLE           Df Model:
26
Date:                  Sun, 10 Aug 2025    Pseudo R-squ.:            0.063
49
Time:                  18:06:39          Log-Likelihood:           -291
6.3
converged:              True           LL-Null:                   -311
4.0
Covariance Type:        nonrobust        LLR p-value:              1.102e-
67
=====
```

```
=====
=====
                                coef      std err          z      P>|z|
-----
[0.025      0.975]
-----
const                        0.5686      0.270      2.106      0.035
0.039      1.098
hhold_ID                     8.644e-06    1.44e-05     0.601      0.548  -
1.96e-05    3.68e-05
discount_original            -0.5308      0.065     -8.182      0.000
-0.658     -0.404
coupon_received              -0.0891      0.072     -1.241      0.215
-0.230      0.052
est_inc                      5.348e-08    2.84e-08     1.884      0.060  -
2.15e-09    1.09e-07
ticket_spend                 -8.811e-05    7.27e-05     -1.213      0.225
-0.000     5.43e-05
incidental_spending          4.694e-06    4.17e-06     1.125      0.261  -
3.49e-06    1.29e-05
park_package                 -0.7065      0.066    -10.771      0.000
-0.835     -0.578
res_to_port                  -0.0035      0.001     -6.293      0.000
-0.005     -0.002
number_children              0.1466      0.025     5.807      0.000
0.097      0.196
oldest_child_age             -0.0453      0.007     -6.577      0.000
-0.059     -0.032
dis_plus_sub                 -0.0387      0.065     -0.591      0.555
-0.167      0.090
cabin_type_Inside            -0.1055      0.114     -0.925      0.355
-0.329      0.118
cabin_type_Oceanview         -0.0362      0.102     -0.354      0.723
-0.237      0.164
cabin_type_Verandah          -0.0544      0.094     -0.579      0.562
-0.239      0.130
```

season_of_year_Spring	0.1416	0.089	1.589	0.112
-0.033 0.316				
season_of_year_Summer	-0.0168	0.089	-0.189	0.850
-0.191 0.157				
season_of_year_Winter	0.0522	0.090	0.583	0.560
-0.123 0.228				
cruise_theme_Princess/Frozen	0.1160	0.094	1.229	0.219
-0.069 0.301				
cruise_theme_Star Wars	0.0888	0.108	0.820	0.412
-0.123 0.301				
cruise_theme_prin_fro	-0.0422	0.109	-0.387	0.699
-0.256 0.172				
cruise_theme_starwars	0.0811	0.108	0.752	0.452
-0.130 0.292				
embark_port_Copenhagen	-0.0254	0.107	-0.238	0.812
-0.234 0.184				
embark_port_New York	0.0579	0.108	0.536	0.592
-0.154 0.270				
embark_port_Port Canaveral	0.0881	0.107	0.827	0.408
-0.121 0.297				
embark_port_San Diego	0.3953	0.109	3.637	0.000
0.182 0.608				
embark_port_Vancouver	0.7390	0.108	6.818	0.000
0.527 0.951				

=====

=====

In [29]: *# Step 6: Select Significant Predictors*

```
# Choose variables with p < 0.05 from Iteration 1
significant_vars = result1.pvalues[result1.pvalues < 0.05].index.tolist()

# Remove the constant if it's in the list
if 'const' in significant_vars:
    significant_vars.remove('const')

print("Significant variables:", significant_vars)
```

Significant variables: ['discount_original', 'park_package', 'res_to_port', 'number_children', 'oldest_child_age', 'embark_port_San Diego', 'embark_port_Vancouver']

In [30]: *# Iteration 2: Logistic Regression (Significant Predictors Only)*

```
X_train_sig = X_train[significant_vars]
X_test_sig = X_test[significant_vars]

logit_model2 = sm.Logit(y_train, sm.add_constant(X_train_sig))
result2 = logit_model2.fit()
print(result2.summary())
```

Optimization terminated successfully.

Current function value: 0.650862

Iterations 5

Logit Regression Results

=====					
==					
Dep. Variable:	return_1065	No. Observations:	45		
00					
Model:	Logit	Df Residuals:	44		
92					
Method:	MLE	Df Model:			
7					
Date:	Sun, 10 Aug 2025	Pseudo R-squ.:	0.059		
44					
Time:	18:08:43	Log-Likelihood:	-292		
8.9					
converged:	True	LL-Null:	-311		
4.0					
Covariance Type:	nonrobust	LLR p-value:	5.844e-		
76					
=====					
=====					
		coef	std err	z	P> z
5	0.975]				[0.02

const		0.5180	0.111	4.661	0.000
0	0.736				0.30
discount_original		-0.5262	0.064	-8.176	0.000
2	-0.400				-0.65
park_package		-0.7017	0.065	-10.764	0.000
9	-0.574				-0.82
res_to_port		-0.0035	0.001	-6.353	0.000
5	-0.002				-0.00
number_children		0.1446	0.025	5.764	0.000
5	0.194				0.09
oldest_child_age		-0.0453	0.007	-6.611	0.000
9	-0.032				-0.05
embark_port_San Diego		0.3571	0.087	4.124	0.000
7	0.527				0.18
embark_port_Vancouver		0.7124	0.086	8.255	0.000
3	0.881				0.54
=====					
=====					

```
In [31]: # Step 7: Evaluate Final Model
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Predict probabilities & classes
y_pred_prob = result2.predict(sm.add_constant(X_test_sig))
y_pred = (y_pred_prob >= 0.5).astype(int)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Confusion matrix
```

```
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

# Classification report
print(classification_report(y_test, y_pred))
```

Accuracy: 0.6073333333333333

Confusion Matrix:

[[1022 520]					
[658 800]]					
	precision	recall	f1-score	support	
0	0.61	0.66	0.63	1542	
1	0.61	0.55	0.58	1458	
accuracy			0.61	3000	
macro avg		0.61	0.61	0.61	3000
weighted avg		0.61	0.61	0.61	3000

Logistic Regression Model Conclusion

Based on our logistic regression model results, there are a few clear takeaways that Disney can use to improve how it predicts and encourages return cruise bookings. We found that certain factors have a stronger influence on the likelihood of a household booking another cruise within three years. For example, customers who booked a park package with their cruise were significantly less likely to return. This could mean that the combined package experience feels like a “one-time splurge” rather than something people repeat often. Likewise, a greater distance from home to the embarkation port slightly reduces the chance of returning, which is expected since travel logistics can be more difficult or costly for these customers.

We also saw positive effects from specific factors. Families with more children were more likely to book another cruise, which may be linked to the value and appeal of Disney’s family-oriented activities. Additionally, some embarkation ports, such as San Diego and Vancouver, showed higher return rates compared to others. This may be due to customer demographics, local marketing, or the convenience and experience associated with those ports. These findings can help Disney target marketing or tailor promotions more effectively by port location.

Our model’s accuracy was about 61%, meaning it correctly predicted whether a household would return about 6 out of 10 times. This is better than random guessing but shows there is still room to grow. The accuracy is limited because we may not have all the information that influences a customer’s decision to return, such as satisfaction ratings, loyalty program status, or detailed onboard spending habits. It also reflects that the current approach is a simpler model, which may miss more complex patterns in the data.

In conclusion, the model gives us a useful starting point for marketing strategies. We recommend focusing on customers who did not purchase a park package, those living closer to embarkation ports, and families with multiple children, as they are more likely to return. At the same time, adding more customer data and trying more advanced modeling methods could improve our prediction power. While the model is not perfect, it offers actionable insights that can guide Disney's customer retention planning.

VII. A/B Testing

```
In [18]: import itertools
from scipy import stats

df_pb = pd.read_csv("../AD 652 Final Project/popcorn_buckets.csv")
df_pb.columns = [c.strip().lower() for c in df_pb.columns]
df_pb.head()
```

```
Out[18]:
```

	location	bucket	q1_sales_usd
0	Disneyland Paris	Cinderella	139413.1
1	Disneyland Paris	Star_Wars	138890.4
2	Disneyland Paris	Mermaid	117696.4
3	Disneyland Paris	MickeySuit	176821.9
4	Disneyland Paris	MickeyEars	168442.1

```
In [19]: # --- Helper to detect columns ---
def find_col(substrings, prefer_numeric=False):
    # exact match
    for s in substrings:
        if s in df_pb.columns:
            return s
    # substring match
    hits = [c for c in df_pb.columns if any(s in c for s in substrings)]
    if prefer_numeric:
        hits = [c for c in hits if pd.api.types.is_numeric_dtype(df_pb[c])]
    return hits[0] if hits else None
```

```
In [20]: # Auto-detect columns
loc_col = find_col(["location", "park", "resort", "site", "region"])
bucket_col = find_col(["bucket", "bucket_type", "bucketstyle", "bucket_style", "
rev_col = find_col(["revenue", "sales", "gross", "total"], prefer_numeric=True)

print("Detected columns:")
print(" location:", loc_col)
print(" bucket :", bucket_col)
print(" revenue :", rev_col)
```

Detected columns:
 location: location
 bucket : bucket
 revenue : q1_sales_usd

```
In [21]: # --- Aggregate one revenue value per (park, bucket) ---
agg = (df_pb.groupby([loc_col, bucket_col], dropna=False)[rev_col]
        .sum()
        .reset_index())
```

```
In [22]: # --- Pivot: parks as rows, buckets as columns ---
wide = agg.pivot(index=loc_col, columns=bucket_col, values=rev_col)
```

```
In [23]: # Keep only buckets with >=2 parks of data
valid_buckets = [b for b in wide.columns if wide[b].notna().sum() >= 2]
wide = wide[valid_buckets]

if len(valid_buckets) < 2:
    raise ValueError("Not enough buckets with >=2 observations to compare.")
```

```
In [24]: # --- Pairwise Welch t-tests with Bonferroni correction ---
pairs = list(itertools.combinations(valid_buckets, 2))
m = len(pairs)

rows = []
for a, b in pairs:
    xa = wide[a].dropna().values
    xb = wide[b].dropna().values
    if len(xa) >= 2 and len(xb) >= 2:
        t, p = stats.ttest_ind(xa, xb, equal_var=False)
        p_b = min(p * m, 1.0) # Bonferroni adjust
        rows.append({
            "bucket_a": a, "bucket_b": b,
            "mean_a": float(np.mean(xa)), "mean_b": float(np.mean(xb)),
            "diff_mean": float(np.mean(xa) - np.mean(xb)),
            "t_stat": float(t), "p_value": float(p), "p_bonf": float(p_b),
            "significant_95%": p_b < 0.05,
            "n_a": int(len(xa)), "n_b": int(len(xb))
        })

ab_results = pd.DataFrame(rows).sort_values("p_bonf") if rows else pd.DataFrame()
print("\nA/B Test Results (Bonferroni-adjusted):")
display(ab_results)
```

A/B Test Results (Bonferroni-adjusted):

	bucket_a	bucket_b	mean_a	mean_b	diff_mean	t_stat
12	MickeySuit	RedWhite	167465.233333	96923.400000	70541.833333	11.714652
10	MickeyEars	RedWhite	154566.350000	96923.400000	57642.950000	8.955610
6	Mermaid	MickeySuit	123628.983333	167465.233333	-43836.250000	-8.879970
3	Cinderella	RedWhite	136139.850000	96923.400000	39216.450000	6.993984
14	RedWhite	Star_Wars	96923.400000	139564.566667	-42641.166667	-6.509617
2	Cinderella	MickeySuit	136139.850000	167465.233333	-31325.383333	-5.906247
5	Mermaid	MickeyEars	123628.983333	154566.350000	-30937.366667	-5.692411
7	Mermaid	RedWhite	123628.983333	96923.400000	26705.583333	5.075998
13	MickeySuit	Star_Wars	167465.233333	139564.566667	27900.666667	4.433789
1	Cinderella	MickeyEars	136139.850000	154566.350000	-18426.500000	-3.193247
0	Cinderella	Mermaid	136139.850000	123628.983333	12510.866667	2.829593
8	Mermaid	Star_Wars	123628.983333	139564.566667	-15935.583333	-2.861301
11	MickeyEars	Star_Wars	154566.350000	139564.566667	15001.783333	2.242153
9	MickeyEars	MickeySuit	154566.350000	167465.233333	-12898.883333	-2.089222
4	Cinderella	Star_Wars	136139.850000	139564.566667	-3424.716667	-0.580725

```
In [25]: # Winners/losers summary
if not ab_results.empty and "significant_95%" in ab_results.columns:
    winners = (ab_results[ab_results["significant_95%"] & (ab_results["diff_m
    .groupby("bucket_a")["bucket_b"].count().sort_values(ascending
    losers = (ab_results[ab_results["significant_95%"] & (ab_results["diff_m
    .groupby("bucket_b")["bucket_a"].count().sort_values(ascending
    print("\nBuckets with the most significant wins:")
    display(winners)
    print("Buckets with the most significant losses:")
    display(losers)
```

```

Buckets with the most significant wins:
bucket_a
MickeySuit      2
Cinderella      1
Mermaid         1
MickeyEars      1
Name: bucket_b, dtype: int64
Buckets with the most significant losses:
bucket_b
MickeySuit      2
MickeyEars      1
Star_Wars       1
Name: bucket_a, dtype: int64

```

AB Test Results

We ran Welch's two-sample t-tests on park-level revenues for every possible head-to-head popcorn bucket matchup, applying a Bonferroni correction for multiple comparisons at the 95% confidence level. The results show several statistically significant differences. MickeySuit was the strongest performer, with significant wins over RedWhite (**+\$70,541**) and MickeyEars (**+\$57,642**). Cinderella also performed well, significantly outperforming RedWhite (**+\$39,216**). Mermaid recorded significant wins against MickeyEars (**+\$26,705**) and Star_Wars (**+\$27,706**). On the other hand, RedWhite and Star_Wars frequently appeared on the losing side of significant matchups, indicating weaker performance. Based on this analysis, Disney should focus production and marketing on high-performing styles like MickeySuit, Cinderella, and Mermaid, while considering reducing or retiring low-performing styles such as RedWhite and Star_Wars to maximize sales potential.

VIII. Strategic Analysis Memo

This strategic analysis is intended to take a deep dive into Disney Park's business path and identify future opportunities to cultivate more success. This report will pinpoint areas that Disney has demonstrated strengths and evaluate areas that show weaknesses. This will help define the criteria that should be used in new project consideration.

Disney has 12 parks worldwide, with high success rates like Magic Kingdom drawing in 17 million visitors per year compared to Universal's Orlando only at 5 million annually. The business demonstrated strong resilience during the 2008–2009 recession and has maintained a consistent attendance rate even up against serious cost sensitivity. While the industry shows high levels of competition against businesses like Universal Studios, Disney has been able to maintain a competitive advantage. The popularity of their destinations aligns with tourism trends, showing Orlando park as their top performer.

Historically, Disney has gone through various eras and changes over the years. In the 1950's the world saw the Walt Disney Era, which was very family-focused and heavily valued the guest experience. The "Disney Look" was the key to success, and the park drew in 1 million guests in under 2 months, generating **\$24.5M** of revenue in just the first year. After years of ongoing success, the business entered the Eisner Era in 1984. They expanded their business model to offer more adult-focused experiences like thrilling rides and Disney cruises, resulting in revenue growth from **\$1B** to **\$9B**. Come 2005, the business kicked off the Iger Era which zoned in on creativity, leading to the acquisition of Pixar. With a refocus on business strategy to combat past failures, the company shifted back toward family-focused experiences while targeting high-value expansion opportunities.

In 2010, Disney Parks generated **\$11B** in revenue, representing 30% of the company's total revenue, along with **\$1.4B** in operating income. The newly appointed Chairman of Walt Disney Parks, Tom Staggs, needs a solid strategy to achieve strong capital allocation across various high-cost projects while fighting the increasing competitive presence of new high-demand experiences like Universal's Wizarding World of Harry Potter. Universal saw a 20% increase in attendance along with a 41% increase in revenue, showing significant growth in the market. Even with strong competition and in the face of a recession, Disney still invested **\$1B** per year, with a 25% increase in domestic visitors over the past eight years.

Currently, Disney faces a number of strategic challenges that will require proper analysis to stay on track. Firstly, maintaining a competitive advantage with the entry of new high-demand attractions being introduced by Universal. Furthermore, Disney must carefully choose projects that will continue to build their brand on a **\$10B+** revenue base. The business demonstrates internal strengths in being the global attendance leader, having strong brand equity, Imagineering, and cross-business collaboration. However, Disney shows weaknesses due to high capital needs, varying results in non-park-focused ventures, and complex operations to manage. Externally, they face economic challenges due to the reliance on discretionary spending on travel, which can be volatile. Socially, they are up against an aging population that is showing decreases in the amount of children per household and different interests with new generations.

There are a number of strategic opportunities that can provide increased value for the business. First, "Digital Overhaul" and introducing the "MagicBand", which would offer RFID wristbands for park entry, hotel access, payments, and ride reservations. This would mean faster throughput than the current data which is showing 8–10k guests per hour. The potential benefits of this project include increased spending in the park, improved guest satisfaction, and very valuable behavioral data. However, with a project cost predicted at **\$1B**, the risks are a large upfront cost, complex technology adoption, and integration issues. Another strategic project to consider is the Hawaiian Hotel, Aulani. Boasting a capacity of 360 hotel rooms and 480 timeshares, the project could

bring Disney's brand into Hawaii's tourism market which they could leverage to improve their hospitality reputation. With a cost of **\$800M**, there could be risk because there are no rides or classic Disney attractions that are associated with their brand. It may require further expansion to match destinations like the Atlantis Bahamas. Finally, Disney could look at intellectual property in park expansions, like Pandora – The World of Avatar. With projects like this, Disney could better compete with the strength of attractions like Harry Potter World and increase park appeal. However, this will require major capital investment and is dependent on the continuous popularity of IP-based entertainment.

In order to determine which project demonstrates the best opportunity, we will use the following criteria to assess them: financial ROI and payback period, brand fit, competitive advantage potential, execution and operational risk, and the impact on guest satisfaction and attendance. Given these factors, the primary recommendation would be to move forward with MagicBand to modernize the guest experience, improve operations, and gather valuable data for long-term revenue growth. As a second focus, Disney should proceed with expanding Aulani using a phased rollout approach. This can diversify revenue sources outside of the park gates; however, it will be crucial to expand amenities and entertainment over time to ensure it competes with destination resorts like Atlantis Bahamas. Finally, Disney should take an ongoing approach in investing in IP-based technology to protect market share against rivals. Using a more balanced and year-by-year strategy ensures both immediate financial gains and long-term future relevance.

In terms of implementing these projects, the business should use different strategies for each rollout. Firstly, for the MagicBand, they should select one park to test adoption before full-scale implementation. Additionally, they should clearly communicate the update to visitors using emails, tutorials, and on-site help desks to ensure smooth onboarding. For Aulani, the first phase should be a core hotel and timeshare units that are family-focused and provide a premium service vacation. In phase two, they should begin adding in entertainment amenities like water park features, cultural experiences, and possibly themed attractions to test popularity. For the IP expansion, they must identify and focus on franchises with proven multi-decade relevance like Frozen, Marvel, or Star Wars. In rolling these out, it is recommended to avoid multiple launches in the same year in order to spread marketing impact over time.

IX. Final Conclusion

Our analysis shows that Disney's strength can be seen through its world-class brand, iconic imagineering, and their ability to leverage data-driven insights to make more strategic decisions. From segmentation and targeting to conjoint analysis and A/B testing, we have seen each method produce its own valuable insights and actionable recommendations. Specifically, focusing on luxury travellers, optimizing in-park

amenities, and strategic merchandising based on popularity have all proven to be highly rewarding areas of focus. For Disney, our findings reinforce the value in aligning product, pricing, and marketing strategies with specific audience segments, ensuring that investments and creative projects are successful with those target markets.

Just as important as the results is the process that led to them. By combining exploratory data analysis, statistical analysis, and business strategy frameworks, our team has proven that these analytical tools are crucial in decision making. Additionally, by using both qualitative and quantitative judgment we have demonstrated that using a cross-functional approach can best create a vision for a path forward. For Disney, the biggest takeaway is that sustainable growth will depend on their ability to align their imaginative experiences with data backed strategies in order to adapt in a highly evolving industry.